

- **数组 / 字符串**
- 88 合并两个有序数组
- 27 移除元素
- 26 删除有序数组中的重复项
- 80 删除有序数组中的重复项 II
- 169 多数元素
- 189 轮转数组
- 121 买卖股票的最佳时机
- 55 跳跃游戏
- 45 跳跃游戏 II
- 13 罗马数字转整数
- 58 最后一个单词的长度
- 14 最长公共前缀
- 28 找出字符串中第一个匹配项的下标
- **双指针**
- 125 验证回文串
- 392 判断子序列
- 15 三数之和
- **滑动窗口**
- 209 长度最小的子数组
- 3 无重复字符的最长子串
- **矩阵**
- 无
- **哈希表**
- 1 两数之和
- **区间**
- 无
- **栈**
- 20 有效的括号
- 155 最小栈
- **链表**
- 141 环形链表
- 2 两数相加
- 21 合并两个有序链表
- 19 删除链表的倒数第 N 个结点
- 146 LRU 缓存
- **二叉树**
- 无
- **二叉树层次遍历**
- 无

- **二叉搜索树**
- [无](#)
- **图**
- [无](#)
- **图的广度优先搜索**
- [无](#)
- **字典树**
- [无](#)
- **回溯**
- [无](#)
- **分治**
- [无](#)
- **Kadane 算法**
- [无](#)
- **二分查找**
- [35 搜索插入位置](#)
- [34 在排序数组中查找元素的第一个和最后一个位置](#)
- **堆**
- [无](#)
- **位运算**
- [136 只出现一次的数字](#)
- **数学**
- [9 回文数](#)
- [66 加一](#)
- **一维动态规划**
- [70 爬楼梯](#)
- **多维动态规划**
- [5 最长回文子串](#)
- **Other**
- [83 删除排序链表中的重复元素](#)
- [24 两两交换链表中的节点](#)
- [206 反转链表](#)

数组 / 字符串

88 合并两个有序数组

```
class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        p1, p2, p = m - 1, n - 1, m + n - 1
        while p2 >= 0: # nums2 还有要合并的元素
            # 如果 p1 < 0, 那么走 else 分支, 把 nums2 合并到 nums1 中
            if p1 >= 0 and nums1[p1] > nums2[p2]:
                nums1[p] = nums1[p1] # 填入 nums1[p1]
                p1 -= 1
            else:
                nums1[p] = nums2[p2] # 填入 nums2[p2]
                p2 -= 1
            p -= 1 # 下一个要填入的位置
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/merge-sorted-array/solutions/2385610/dao-xu-shuang-zhi-zhen-wei-shi-yao-dao-x-xxkp/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设 `nums1 = [1,2,3,0,0,0]`, `m = 3`, `nums2 = [2,5,6]`, `n = 3`, 请将这段代码转换为swift版本, 并给出完整调用代码。

下面是将你提供的 Python 代码转换为 **Swift** 的版本, 并附上完整的调用示例。

✅ Swift 版本实现

```
class Solution {  
    func merge(_ nums1: inout [Int], _ m: Int, _ nums2: inout [Int], _ n: Int) {  
        var p1 = m - 1  
        var p2 = n - 1  
        var p = m + n - 1  
  
        while p2 >= 0 {  
            if p1 >= 0 && nums1[p1] > nums2[p2] {  
                nums1[p] = nums1[p1]  
                p1 -= 1  
            } else {  
                nums1[p] = nums2[p2]  
                p2 -= 1  
            }  
            p -= 1  
        }  
    }  
}
```

注意：在 Swift 中，数组是值类型，所以如果要修改传入的数组，必须使用 `inout` 参数。

🖋 完整调用示例

```
import Foundation  
  
var nums1 = [1, 2, 3, 0, 0, 0]  
let m = 3  
var nums2 = [2, 5, 6]  
let n = 3  
  
let solution = Solution()  
solution.merge(&nums1, m, &nums2, n)  
  
print(nums1) // 输出: [1, 2, 2, 3, 5, 6]
```

🔍 说明

- 初始时 `nums1` 的长度为 `m + n`，后 `n` 个位置是预留的 `0`。

- 算法采用 **从后往前** 的双指针策略，避免覆盖尚未处理的元素。
- 时间复杂度： $O(m + n)$
- 空间复杂度： $O(1)$

如有更多 LeetCode 题目需要转换或解释，欢迎继续提问！

27 移除元素

```
class Solution:
    def removeElement(self, nums: List[int], val: int) -> int:
        stack_size = 0
        for x in nums:
            if x != val:
                nums[stack_size] = x # 把 x 入栈
                stack_size += 1
        return stack_size
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/remove-element/solutions/2802809/jian-dan-ti-jian-dan-zuo-pythonjavaccgoj-72bn/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `nums = [3,2,2,3]`，`val = 3`，请将这段代码转换为swift版本，并给出完整调用代码。

以下是将你提供的 Python 代码转换为 **Swift** 版本的完整实现，包括调用示例：

✅ Swift 版本实现

```
class Solution {  
    func removeElement(_ nums: inout [Int], _ val: Int) -> Int {  
        var stackSize = 0  
        for x in nums {  
            if x != val {  
                nums[stackSize] = x // 相当于“入栈”  
                stackSize += 1  
            }  
        }  
        return stackSize  
    }  
}
```

🔍 注意：Swift 中数组默认是值类型（Array），若要在函数内修改传入的数组，必须使用 `inout` 参数。

🖋️ 完整调用示例

```
// 输入数据  
var nums = [3, 2, 2, 3]  
let val = 3  
  
// 调用方法  
let k = Solution().removeElement(&nums, val)  
  
// 输出结果  
print("k = \(k)") // 输出: k = 2  
print("nums = \(nums)") // 输出: nums = [2, 2, 2, 3]（后两位无所谓）  
  
// 根据题目要求，只需关注前 k 个元素  
print("有效部分: \(Array(nums[0..  
k]))") // 输出: 有效部分: [2, 2]
```

🖋️ 说明

- 题目只要求 **前 k 个元素** 是移除 `val` 后的结果，后面的元素可以是任意值。
- 上述代码完全符合 LeetCode 的原地修改（in-place）要求，时间复杂度 **O(n)**，空间复杂度 **O(1)**。

如有需要，也可以封装成一个独立函数而非类方法。欢迎继续提问！

26 删除有序数组中的重复项

```
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        k = 1
        for i in range(1, len(nums)):
            if nums[i] != nums[i - 1]: # nums[i] 不是重复项
                nums[k] = nums[i] # 保留 nums[i]
                k += 1
        return k
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/remove-duplicates-from-sorted-array/solutions/2807162/gen-zhao-wo-guo-yi-bian-shi-li-2ni-jiu-m-rvyk/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设 `nums = [1,1,2]`，请将这段代码转换为swift版本，并给出完整调用代码。

以下是将你提供的 Python 代码转换为 **Swift** 版本的实现，并附带完整的调用示例：

✅ Swift 实现（LeetCode 风格）

```
class Solution {
    func removeDuplicates(_ nums: inout [Int]) -> Int {
        guard !nums.isEmpty else { return 0 }

        var k = 1
        for i in 1..
```

✅ 完整调用示例

```
// 测试用例
var nums = [1, 1, 2]

let solution = Solution()
let k = solution.removeDuplicates(&nums)

print("k = \(k)")           // 输出: k = 2
print("nums = \(nums)")     // 输出: nums = [1, 2, 2] (前 k 个元素是去重后的结果)
print("去重后的数组 = \(Array(nums[0..
```

说明:

- Swift 中数组默认是值类型，要修改原数组需使用 `inout` 参数并传入时加 `&`。
- LeetCode 要求 **原地修改** 数组，并返回新长度 `k`，前 `k` 个元素即为去重结果。
- 虽然数组后面可能仍有旧值（如 `[1, 2, 2]`），但只要前 `k` 个正确即可。

如有其他测试用例或想封装成独立函数，也可以告诉我！

80 删除有序数组中的重复项 II

```
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        stack_size = 2 # 栈的大小，前两个元素默认保留
        for i in range(2, len(nums)):
            if nums[i] != nums[stack_size - 2]: # 和栈顶下方的元素比较
                nums[stack_size] = nums[i] # 入栈
                stack_size += 1
        return min(stack_size, len(nums))
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/remove-duplicates-from-sorted-array-ii/solutions/3060042/yong-zhan-si-kao-yuan-di-shi-xian-python-zw8l/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `nums = [1,1,1,2,2,3]`，请将这段代码转换为swift版本，并给出完整调用代码。

下面是将你提供的 Python 代码转换为 **Swift** 的完整实现，包括函数定义和调用示例：

✅ Swift 版本实现

```
import Foundation

class Solution {
    func removeDuplicates(_ nums: inout [Int]) -> Int {
        // 如果数组长度小于等于2，直接返回原长度（无需删除）
        if nums.count <= 2 {
            return nums.count
        }

        var stackSize = 2 // 前两个元素默认保留

        for i in 2..
```

输出结果

运行上述代码后，输出为：

新长度：5

修改后的数组前 5 个元素：[1, 1, 2, 2, 3]

这与题目要求完全一致。

补充说明

- Swift 中数组是值类型，要原地修改必须使用 `inout` 参数，并在调用时加 `&`。
- 由于循环从 `i = 2` 开始，且 `stackSize` 初始为 2，因此当 `nums.count <= 2` 时可直接返回，避免越界。
- `min(stackSize, nums.count)` 在逻辑上其实不需要，因为 `stackSize` 最大不会超过 `nums.count`（每次只在合法范围内赋值），所以直接返回 `stackSize` 即可。

如有需要支持通用保留 `k` 个重复项的版本，也可以进一步扩展。

169 多数元素

```
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        ans = hp = 0
        for x in nums:
            if hp == 0: # x 是初始擂主，生命值为 1
                ans, hp = x, 1
            else: # 比武，同门加血，否则扣血
                hp += 1 if x == ans else -1
        return ans
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/majority-element/solutions/3744717/on-mo-er-tou-piao-fa-yan-jin-zheng-ming-ww1zv/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `nums = [3,2,3]`，请将这段代码转换为swift版本，并给出完整调用代码。

以下是将你提供的 Python 代码转换为 **Swift** 的版本，并附带完整的调用示例：

✅ Swift 版本实现（摩尔投票法）

```
import Foundation

class Solution {
    func majorityElement(_ nums: [Int]) -> Int {
        var ans = 0
        var hp = 0 // hp 表示“生命值”或计数器

        for x in nums {
            if hp == 0 {
                ans = x
                hp = 1
            } else {
                hp += (x == ans) ? 1 : -1
            }
        }

        return ans
    }
}

// ===== 调用示例 =====
let nums = [3, 2, 3]
let solution = Solution()
let result = solution.majorityElement(nums)
print("多数元素是: \(result)") // 输出: 多数元素是: 3
```

🔍 说明：

- `hp` 是“hit points”（生命值）的缩写，在摩尔投票算法中用于记录当前候选元素的“净支持票数”。
- 当 `hp == 0` 时，更换候选者。
- 由于题目保证存在多数元素（出现次数 $> n/2$ ），因此最后 `ans` 一定是正确答案，无需二次验证（若题目不保证，则需再遍历一次验证）。

这段代码在 LeetCode 的 [169. 多数元素](#) 题目中完全适用。

189 轮转数组

```
class Solution {  
    public void rotate(int[] nums, int k) {  
        int n = nums.length;  
        int[] newArr = new int[n];  
        for (int i = 0; i < n; ++i) {  
            newArr[(i + k) % n] = nums[i];  
        }  
        System.arraycopy(newArr, 0, nums, 0, n);  
    }  
}
```

作者：力扣官方题解

链接：<https://leetcode.cn/problems/rotate-array/solutions/551039/xuan-zhuan-shu-zu-by-leetcode-solution-nipk/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `nums = [1,2,3,4,5,6,7]`，`k = 3`，请将这段代码转换为swift版本，并给出完整调用代码。

以下是将你提供的 Java 代码转换为 **Swift** 的版本，并附带完整的调用示例：

```

class Solution {
    func rotate(_ nums: inout [Int], _ k: Int) {
        let n = nums.count
        var newArr = [Int](repeating: 0, count: n)
        for i in 0..

```

```

// 调用示例
var nums = [1, 2, 3, 4, 5, 6, 7]
let k = 3
let solution = Solution()
solution.rotate(&nums, k)
print(nums) // 输出: [5, 6, 7, 1, 2, 3, 4]

```

说明：

- Swift 中数组是值类型，因此如果要在函数中修改传入的数组，需要使用 `inout` 参数。
- 使用 `&nums` 来传递可变引用。
- `newArr` 初始化为长度为 `n`、元素全为 `0` 的数组。
- 最后直接赋值 `nums = newArr` 即可完成原地效果（虽然底层是替换整个数组）。

这段代码逻辑与原始 Java 版本完全一致，适用于 LeetCode 的 Swift 环境。

121 买卖股票的最佳时机

```

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        ans = 0
        min_price = prices[0]
        for p in prices:
            ans = max(ans, p - min_price)
            min_price = min(min_price, p)
        return ans

```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/best-time-to-buy-and-sell-stock/solutions/2464650/mei-ju-mai-chu-jie-ge-wei-hu-mai-ru-de-z-02ud/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `prices = [7,1,5,3,6,4]`，请将这段代码转换为swift版本，并给出完整调用代码。

以下是将你提供的 Python 代码转换为 **Swift** 的完整版本，包括类定义和调用示例：

```
import Foundation

class Solution {
    func maxProfit(_ prices: [Int]) -> Int {
        var ans = 0
        var minPrice = prices[0]
        for p in prices {
            ans = max(ans, p - minPrice)
            minPrice = min(minPrice, p)
        }
        return ans
    }
}

// 调用示例
let prices = [7, 1, 5, 3, 6, 4]
let solution = Solution()
let result = solution.maxProfit(prices)
print("最大利润为: \(result)") // 输出：最大利润为： 5
```

说明：

- Swift 中使用 `max` 和 `min` 函数与 Python 类似。
- 数组通过值传递（但内部是 copy-on-write，效率高）。
- `prices[0]` 假设输入非空，符合题目约束（`1 <= prices.length`）。

这段代码的时间复杂度为 **$O(n)$** ，空间复杂度为 **$O(1)$** ，与原 Python 版本一致。

55 跳跃游戏

```
class Solution:
    def canJump(self, nums: List[int]) -> bool:
        mx = 0
        for i, jump in enumerate(nums):
            if i > mx: # 无法到达 i
                return False
            mx = max(mx, i + jump) # 从 i 最右可以跳到 i + jump
        if mx >= len(nums) - 1: # 可以跳到 n-1
            return True
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/jump-game/solutions/2798996/liang-chong-li-jie-fang-shi-wei-hu-zui-y-q67s/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `nums = [2,3,1,1,4]`，请将这段代码转换为swift版本，并给出完整调用代码。

以下是将你提供的 Python 代码转换为 **Swift** 的完整版本，包括类定义和调用示例：

```

class Solution {
    func canJump(_ nums: [Int]) -> Bool {
        var mx = 0
        let n = nums.count

        for (i, jump) in nums.enumerated() {
            if i > mx {
                return false
            }
            mx = max(mx, i + jump)
            if mx >= n - 1 {
                return true
            }
        }

        return true // 实际上不会走到这里，因为循环中已处理所有情况
    }
}

// 调用示例
let solution = Solution()
let nums = [2, 3, 1, 1, 4]
let result = solution.canJump(nums)
print(result) // 输出: true

```

说明：

- Swift 中使用 `[Int]` 表示整数数组。
- `enumerated()` 方法用于同时获取索引和元素，类似于 Python 的 `enumerate`。
- `max(_:_:_:)` 是 Swift 标准库函数。
- 最后一行 `return true` 虽然逻辑上不会执行（因为如果能遍历完所有元素，说明一定可以到达终点），但为了满足 Swift 编译器对非空返回的要求而保留。

你可以将这段代码粘贴到 Xcode Playground 或 LeetCode 的 Swift 编辑器中运行。

45 跳跃游戏 II

```
class Solution:
    def jump(self, nums: List[int]) -> int:
        ans = 0
        cur_right = 0 # 已建造的桥的右端点
        next_right = 0 # 下一座桥的右端点的最大值
        for i in range(len(nums) - 1):
            # 遍历的过程中，记录下一座桥的最远点
            next_right = max(next_right, i + nums[i])
            if i == cur_right: # 无路可走，必须建桥
                cur_right = next_right # 建桥后，最远可以到达 next_right
                ans += 1
        return ans
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/jump-game-ii/solutions/2926993/tu-jie-yi-zhang-tu-miao-dong-tiao-yue-yo-h2d4/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `nums = [2,3,1,1,4]`，请将这段代码转换为swift版本，并给出完整调用代码。

以下是将你提供的 Python 代码转换为 **Swift** 的完整版本，包括类定义和调用示例：

```

import Foundation

class Solution {
    func jump(_ nums: [Int]) -> Int {
        var ans = 0
        var curRight = 0      // 当前跳跃能到达的最远右边界
        var nextRight = 0     // 下一次跳跃能到达的最远右边界

        // 注意：只遍历到倒数第二个元素，因为到达最后一个位置就结束了
        for i in 0..

```

说明：

- Swift 中数组使用 `[Int]` 类型。
- `for i in 0.. 对应 Python 的 for i in range(len(nums) - 1)。`
- 逻辑完全一致：贪心策略，每次在当前可达范围内找到下一次能跳得最远的位置。

运行上述代码会输出：

```
最少跳跃次数：2
```

这与 LeetCode 示例一致（从索引 $0 \rightarrow 1 \rightarrow 4$ ）。

13 罗马数字转整数

```
# 单个罗马数字到整数的映射
ROMAN = {
    'I': 1,
    'V': 5,
    'X': 10,
    'L': 50,
    'C': 100,
    'D': 500,
    'M': 1000,
}

class Solution:
    def romanToInt(self, s: str) -> int:
        ans = 0
        for x, y in pairwise(s): # 遍历 s 中的相邻字符
            x, y = ROMAN[x], ROMAN[y]
            # 累加 x 或者 -x, 这里 y 只是用来辅助判断 x 的正负
            ans += x if x >= y else -x
        return ans + ROMAN[s[-1]] # 加上最后一个罗马数字
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/roman-to-integer/solutions/2928945/jian-dan-ti-jiu-you-jian-dan-xie-fa-pyth-egyn/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `s = "III"`，请将这段代码转换为swift版本，并给出完整调用代码。

下面是将你提供的 Python 代码（使用 `pairwise` 遍历相邻字符）转换为 **Swift** 的完整实现，并附上可运行的调用示例。

✅ Swift 版本实现

```
import Foundation

class Solution {
    func romanToInt(_ s: String) -> Int {
        let ROMAN: [Character: Int] = [
            "I": 1,
            "V": 5,
            "X": 10,
            "L": 50,
            "C": 100,
            "D": 500,
            "M": 1000
        ]

        let chars = Array(s)
        var ans = 0

        // 遍历相邻字符对 (chars[i], chars[i+1])
        for i in 0..
```

🔍 说明:

- Swift 没有内置的 `pairwise` 函数，所以我们手动通过索引遍历 `0 ..< count - 1` 来处理相邻字符。
- 将字符串 `s` 转为 `[Character]` 数组便于按索引访问。

- 最后一个字符总是正向加上的，因为没有后续字符可以构成减法规则。

✅ 测试其他样例（可选）：

你可以替换 `input` 为如下值测试：

```
"IV"      // → 4
"IX"      // → 9
"LVIII"   // → 58
"MCMXCIV" // → 1994
```

都会正确输出对应整数。

如需进一步封装或支持错误处理（比如非法字符），也可以扩展，但题目保证输入合法，所以这里保持简洁。

58 最后一个单词的长度

```
class Solution:
    def lengthOfLastWord(self, s: str) -> int:
        i = len(s) - 1
        while s[i] == ' ':
            i -= 1

        j = i - 1
        while j >= 0 and s[j] != ' ':
            j -= 1

        return i - j
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/length-of-last-word/solutions/2787453/liang-chong-xie-fa-shou-xie-xun-huan-ku-nmzli/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `s = "Hello World"`，请将这段代码转换为swift版本，并给出完整调用代码。

以下是将你提供的 Python 代码转换为 **Swift** 的版本，并附上完整的可运行调用代码：

✅ Swift 版本实现

```
class Solution {
    func lengthOfLastWord(_ s: String) -> Int {
        let chars = Array(s) // 将字符串转为字符数组，便于下标访问
        var i = chars.count - 1

        // 跳过末尾的空格
        while i >= 0 && chars[i] == " " {
            i -= 1
        }

        // 此时 i 指向最后一个单词的最后一个字母
        var j = i - 1
        // 向前找到单词的开头（即遇到空格或到头）
        while j >= 0 && chars[j] != " " {
            j -= 1
        }

        return i - j
    }
}
```

✅ 完整调用示例

```
let solution = Solution()
let input = "Hello World"
let result = solution.lengthOfLastWord(input)
print("最后一个单词的长度是: \(result)") // 输出: 5
```

🔍 补充说明

- Swift 中 `String` 不支持直接通过整数下标访问字符（不像 Python），所以需要先转成 `[Character]` 数组。
- 该实现完全复刻了灵茶山艾的逻辑：
 - 从后往前跳过空格；
 - 找到最后一个单词的起始位置；
 - 返回长度 `i - j`。

你可以将上述代码粘贴到 Xcode Playground 或 Swift 项目中直接运行。

14 最长公共前缀

```
class Solution:
    def longestCommonPrefix(self, strs: List[str]) -> str:
        s0 = strs[0]
        for j, c in enumerate(s0): # 从左到右
            for s in strs: # 从上到下
                if j == len(s) or s[j] != c: # 这一列有字母缺失或者不同
                    return s0[:j] # 0 到 j-1 是公共前缀
        return s0
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/longest-common-prefix/solutions/2801713/jian-dan-ti-jian-dan-zuo-pythonjavaccgoj-478q/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `strs = ["flower","flow","flight"]`，请将这段代码转换为swift版本，并给出完整调用代码。

下面是将你提供的 Python 代码转换为 **Swift** 的完整实现，包括函数定义和调用示例：

✅ Swift 版本实现

```
import Foundation

class Solution {
    func longestCommonPrefix(_ strs: [String]) -> String {
        guard !strs.isEmpty else { return "" }

        let s0 = strs[0]
        let s0Array = Array(s0) // 将字符串转为字符数组以便按索引访问

        for (j, c) in s0Array.enumerated() {
            for s in strs {
                let sArray = Array(s)
                if j >= sArray.count || sArray[j] != c {
                    return String(s0Array.prefix(j))
                }
            }
        }

        return s0
    }
}

// ===== 调用示例 =====
let solution = Solution()
let input1 = ["flower", "flow", "flight"]
let result1 = solution.longestCommonPrefix(input1)
print("输入: \(input1)")
print("输出: \"\((result1)\")\" // 应输出 \"fl\"

let input2 = ["dog", "racecar", "car"]
let result2 = solution.longestCommonPrefix(input2)
print("输入: \(input2)")
print("输出: \"\((result2)\")\" // 应输出 ""
```

🔍 说明

- Swift 中 `String` 不支持直接通过整数下标访问字符（不像 Python），所以需要先转成 `[Character]` 数组。
- 使用 `prefix(j)` 可以安全地截取前 `j` 个字符，并转回 `String`。
- 边界情况如空数组已通过 `guard` 处理。

运行结果（预期）

```
输入: ["flower", "flow", "flight"]
输出: "fl"
输入: ["dog", "racecar", "car"]
输出: ""
```

你可以将上述代码粘贴到 Xcode Playground 或任何 Swift 环境中运行验证。

28 找出字符串中第一个匹配项的下标

```
class Solution {
    public int strStr(String ss, String pp) {
        int n = ss.length(), m = pp.length();
        char[] s = ss.toCharArray(), p = pp.toCharArray();
        // 枚举原串的「发起点」
        for (int i = 0; i <= n - m; i++) {
            // 从原串的「发起点」和匹配串的「首位」开始，尝试匹配
            int a = i, b = 0;
            while (b < m && s[a] == p[b]) {
                a++;
                b++;
            }
            // 如果能够完全匹配，返回原串的「发起点」下标
            if (b == m) return i;
        }
        return -1;
    }
}
```

作者：宫水三叶

链接：<https://leetcode.cn/problems/find-the-index-of-the-first-occurrence-in-a-string/solutions/575568/shua-chuan-lc-shuang-bai-po-su-jie-fa-km-tb86/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `haystack = "sadbutsad"`, `needle = "sad"`，请将这段代码转换为swift版本，并给出完整调用代码。

下面是将你提供的 Java 代码（宫水三叶的朴素双指针解法）转换为 **Swift** 版本的完整实现，并附上完整的调用示例。

✅ Swift 版本实现

```
class Solution {
    func strStr(_ haystack: String, _ needle: String) -> Int {
        let n = haystack.count
        let m = needle.count

        // 如果 needle 为空，按题意应返回 0（虽然题目提示长度 >=1，但良好习惯）
        if m == 0 { return 0 }

        let s = Array(haystack)
        let p = Array(needle)

        // 枚举原串的「发起点」
        for i in 0...(n - m) {
            var a = i
            var b = 0

            // 尝试匹配
            while b < m && s[a] == p[b] {
                a += 1
                b += 1
            }

            // 如果完全匹配，返回起始下标
            if b == m {
                return i
            }
        }

        return -1
    }
}
```

✅ 完整调用示例

```
let solution = Solution()

// 示例 1
let haystack1 = "sadbutsad"
let needle1 = "sad"
print(solution.strStr(haystack1, needle1)) // 输出: 0

// 示例 2
let haystack2 = "leetcode"
let needle2 = "leeto"
print(solution.strStr(haystack2, needle2)) // 输出: -1
```

🔍 说明

- Swift 中 `String` 不能直接通过整数下标随机访问（不像 Java 的 `charAt`），所以先转成 `[Character]` 数组以支持 $O(1)$ 索引。
- 循环范围是 $0 \dots (n - m)$ ，使用闭区间，因为 $i == n - m$ 时仍可能完整匹配（例如 `"aaa"` 中找 `"aa"`， $i=1$ 是合法的）。
- 时间复杂度： $O((n - m + 1) * m)$ ，空间复杂度： $O(n + m)$ （因为转成了数组）。

💡 虽然 Swift 标准库有 `haystack.range(of: needle)` 可直接用，但本题考察的是手动实现匹配逻辑，因此不使用内置方法。

如有需要 KMP 或其他优化版本的 Swift 实现，也可以告诉我！

双指针

125 验证回文串

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        i, j = 0, len(s) - 1
        while i < j:
            if not s[i].isalnum():
                i += 1
            elif not s[j].isalnum():
                j -= 1
            elif s[i].lower() == s[j].lower():
                i += 1
                j -= 1
            else:
                return False
        return True
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/valid-palindrome/solutions/3053284/jian-dan-ti-jian-dan-zuo-pythonjavaccgoj-1za0/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `s = "A man, a plan, a canal: Panama"`，请将这段代码转换为swift版本，并给出完整调用代码。

下面是将你提供的 Python 代码转换为 **Swift** 的完整实现，并附带一个完整的调用示例（包括 `main` 函数模拟）：

✅ Swift 版本代码

```
class Solution {
    func isPalindrome(_ s: String) -> Bool {
        let chars = Array(s)
        var i = 0
        var j = chars.count - 1

        while i < j {
            let leftChar = chars[i]
            let rightChar = chars[j]

            if !leftChar.isLetter && !leftChar.isNumber {
                i += 1
            } else if !rightChar.isLetter && !rightChar.isNumber {
                j -= 1
            } else if leftChar.lowercased() == rightChar.lowercased() {
                i += 1
                j -= 1
            } else {
                return false
            }
        }

        return true
    }
}

// MARK: - 测试调用
let solution = Solution()
let input = "A man, a plan, a canal: Panama"
let result = solution.isPalindrome(input)
print("输入字符串: \"\((input)\"")
print("是否为回文串: \"\((result)\"") // 应输出 true
```

🔍 说明:

- Swift 中没有直接的 `isalnum()` 方法，但可以使用：
 - `Character.isLetter`
 - `Character.isNumber`
- 字符串转为 `[Character]` 数组便于通过下标访问。
- 使用 `lowercased()` 来忽略大小写（注意：这是 `String` 方法，所以要把 `Character` 转成 `String` 或直接调用 `.lowercased()` —— Swift 中 `Character` 也支持 `lowercased()` 返回 `String`

ng，但比较时两边都转即可）。

- 上述代码在 **Swift 5+** 环境中可正常运行（如 Xcode Playground、LeetCode Swift 环境等）。

示例输出：

输入字符串: "A man, a plan, a canal: Panama"
是否为回文串: true

如果你是在 LeetCode 上提交，只需保留 `Solution` 类即可；如果本地测试，加上下面的调用部分即可运行。

392 判断子序列

```
class Solution:
    def isSubsequence(self, s: str, t: str) -> bool:
        if not s:
            return True
        i = 0
        for c in t:
            if s[i] == c:
                i += 1
            if i == len(s): # 所有字符匹配完毕
                return True # s 是 t 的子序列
        return False
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/is-subsequence/solutions/2813031/jian-ji-xie-fa-pythonjavaccgojsrust-by-e-mz22/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `s = "abc"`, `t = "ahbgdc"`，请将这段代码转换为swift版本，并给出完整调用代码。

以下是将你提供的 Python 代码转换为 **Swift** 版本的完整实现，并附带调用示例：

✅ Swift 版本代码

```
class Solution {
    func isSubsequence(_ s: String, _ t: String) -> Bool {
        if s.isEmpty {
            return true
        }

        let sArray = Array(s)
        let tArray = Array(t)
        var i = 0

        for c in tArray {
            if sArray[i] == c {
                i += 1
                if i == sArray.count {
                    return true
                }
            }
        }

        return false
    }
}

// 调用示例
let solution = Solution()
let s = "abc"
let t = "ahbgdc"
let result = solution.isSubsequence(s, t)
print(result) // 输出: true
```



说明：

- Swift 中 `String` 不支持直接通过整数下标访问字符（不像 Python），所以我们先将字符串转为 `[Character]` 数组（即 `Array(s)`）。
- 然后使用索引 `i` 遍历 `sArray`，在遍历 `tArray` 的过程中尝试匹配。
- 一旦 `i` 达到 `sArray.count`，说明所有字符都已按顺序匹配成功，返回 `true`。
- 如果遍历完 `t` 仍未完全匹配，则返回 `false`。

🔍 测试用例验证：

- 输入： `s = "abc"` , `t = "ahbgdc"` → 输出： `true`
- 输入： `s = "axc"` , `t = "ahbgdc"` → 输出： `false`
- 输入： `s = ""` , `t = "anything"` → 输出： `true`

你可以将上述代码粘贴到 Xcode Playground 或 LeetCode Swift 编辑器中运行验证。

15 三数之和

```
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        ans = []
        n = len(nums)
        for i in range(n - 2):
            x = nums[i]
            if i > 0 and x == nums[i - 1]: # 跳过重复数字
                continue
            if x + nums[i + 1] + nums[i + 2] > 0: # 优化一
                break
            if x + nums[-2] + nums[-1] < 0: # 优化二
                continue
            j = i + 1
            k = n - 1
            while j < k:
                s = x + nums[j] + nums[k]
                if s > 0:
                    k -= 1
                elif s < 0:
                    j += 1
                else: # 三数之和为 0
                    ans.append([x, nums[j], nums[k]])
                    j += 1
                    while j < k and nums[j] == nums[j - 1]: # 跳过重复数字
                        j += 1
                    k -= 1
                    while k > j and nums[k] == nums[k + 1]: # 跳过重复数字
                        k -= 1
            return ans
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/3sum/solutions/1968332/shuang-zhi-zhen-xiang-bu-ming-bai->

[yi-ge-pno55/](#)

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `nums = [-1,0,1,2,-1,-4]`，请将这段代码转换为swift版本，并给出完整调用代码。

下面是将你提供的 Python 代码（用于 LeetCode 第 15 题“三数之和”）**完整转换为 Swift 版本**的实现，并附上完整的调用示例。

✅ Swift 实现（兼容 LeetCode 环境）

```
import Foundation

class Solution {
    func threeSum(_ nums: [Int]) -> [[Int]] {
        var nums = nums
        nums.sort()
        var ans: [[Int]] = []
        let n = nums.count

        for i in 0 ..< n - 2 {
            let x = nums[i]

            // 跳过重复的 x
            if i > 0 && x == nums[i - 1] {
                continue
            }

            // 优化一：最小的三个数之和已大于 0，不可能有解
            if x + nums[i + 1] + nums[i + 2] > 0 {
                break
            }

            // 优化二：当前 x 与最大的两个数之和仍小于 0，跳过
            if x + nums[n - 2] + nums[n - 1] < 0 {
                continue
            }

            var j = i + 1
            var k = n - 1

            while j < k {
                let s = x + nums[j] + nums[k]
                if s > 0 {
                    k -= 1
                } else if s < 0 {
                    j += 1
                } else {
                    ans.append([x, nums[j], nums[k]])

                    // 跳过重复的 nums[j]
                    j += 1
                    while j < k && nums[j] == nums[j - 1] {
                        j += 1
                    }
                }
            }
        }

        return ans
    }
}
```

```
        // 跳过重复的 nums[k]
        k -= 1
        while k > j && nums[k] == nums[k + 1] {
            k -= 1
        }
    }
}

return ans
}
```

完整调用示例（可在本地 Playground 或 Xcode 中运行）

```
let solution = Solution()
let input = [-1, 0, 1, 2, -1, -4]
let result = solution.threeSum(input)
print(result)
// 输出: [[-1, -1, 2], [-1, 0, 1]]
```

说明

- Swift 中数组是值类型，因此在函数内部对 `nums` 排序不会影响原始输入。
- 使用了与 Python 版本完全一致的双指针 + 去重 + 剪枝逻辑。
- 时间复杂度： $O(n^2)$ ，空间复杂度： $O(1)$ （不计输出空间）。

如需提交到 LeetCode，只需保留 `Solution` 类即可。

滑动窗口

209 长度最小的子数组

```
class Solution:
    def minSubArrayLen(self, target: int, nums: List[int]) -> int:
        n = len(nums)
        ans = n + 1 # 也可以写 inf
        s = left = 0
        for right, x in enumerate(nums): # 枚举子数组右端点
            s += x
            while s - nums[left] >= target: # 尽量缩小子数组长度
                s -= nums[left]
                left += 1 # 左端点右移
            if s >= target:
                ans = min(ans, right-left+1)
        return ans if ans <= n else 0
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/minimum-size-subarray-sum/solutions/1959532/biao-ti-xia-biao-zong-suan-cuo-qing-kan-k81nh/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `target = 7`, `nums = [2,3,1,2,4,3]`，请将这段代码转换为swift版本，并给出完整调用代码。

下面是将你提供的 Python 代码（灵茶山艾府的滑动窗口解法）转换为 **Swift** 的完整版本，包括 `Solution` 类和调用示例：

✅ Swift 版本代码

```
import Foundation

class Solution {
    func minSubArrayLen(_ target: Int, _ nums: [Int]) -> Int {
        let n = nums.count
        var ans = n + 1
        var s = 0
        var left = 0

        for (right, x) in nums.enumerated() {
            s += x
            // 尽量缩小窗口：只要去掉左端点后仍满足 >= target, 就缩小
            while s - nums[left] >= target {
                s -= nums[left]
                left += 1
            }
            // 当前窗口满足条件，更新答案
            if s >= target {
                ans = min(ans, right - left + 1)
            }
        }

        return ans <= n ? ans : 0
    }
}

// ===== 调用示例 =====
let solution = Solution()
let target = 7
let nums = [2, 3, 1, 2, 4, 3]
let result = solution.minSubArrayLen(target, nums)
print("最小长度子数组长度为: \(result)") // 输出: 2
```

🔍 说明:

- 使用了 Swift 的 `enumerated()` 来同时获取索引 `right` 和值 `x`。
- 初始 `ans = n + 1` 是一个比任何合法子数组长度都大的哨兵值。
- 最终通过 `ans <= n ? ans : 0` 判断是否存在有效子数组。

运行结果：

对于输入：

```
target = 7
nums = [2,3,1,2,4,3]
```

输出为：

最小长度子数组长度为：2

对应子数组 [4, 3]，符合题目要求。

你可以将上述代码粘贴到 Xcode Playground 或 Swift 编译环境中直接运行。

3 无重复字符的最长子串

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        ans = left = 0
        cnt = defaultdict(int) # 维护从下标 left 到下标 right 的字符及其出现次数
        for right, c in enumerate(s):
            cnt[c] += 1
            while cnt[c] > 1: # 窗口内有重复字母
                cnt[s[left]] -= 1 # 移除窗口左端点字母
                left += 1 # 缩小窗口
            ans = max(ans, right - left + 1) # 更新窗口长度最大值
        return ans
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/longest-substring-without-repeating-characters/solutions/1959540/xia-biao-zong-suan-cuo-qing-kan-zhe-by-e-iaks/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `s = "abcabcbb"`，请将这段代码转换为swift版本，并给出完整调用代码。

以下是将你提供的 Python 代码转换为 **Swift** 的完整实现，并附带调用示例：

✅ Swift 版本实现

```
import Foundation

class Solution {
    func lengthOfLongestSubstring(_ s: String) -> Int {
        var ans = 0
        var left = 0
        var cnt: [Character: Int] = [:]
        let chars = Array(s) // 将字符串转为字符数组，便于下标访问

        for (right, c) in chars.enumerated() {
            cnt[c, default: 0] += 1

            // 当前字符重复了，收缩窗口
            while cnt[c]! > 1 {
                cnt[chars[left]]! -= 1
                left += 1
            }

            // 更新最长无重复子串长度
            ans = max(ans, right - left + 1)
        }

        return ans
    }
}

// ===== 调用示例 =====
let solution = Solution()
let input = "abcabcbb"
let result = solution.lengthOfLongestSubstring(input)
print("输入: \(input)")
print("最长无重复子串长度: \(result)")
```

🖋️ 输出结果（运行后）：

输入: abcabcbb
最长无重复子串长度: 3

因为 "abc" 是第一个无重复的子串，后面 "abc" 再次出现，但长度仍为 3；最后 "cb" 或 "b" 都更短。

💡 说明:

- Swift 中字典访问需注意可选值，这里使用 `cnt[c, default: 0]` 安全地处理默认值。
- 字符串不能直接按下标访问，所以先转成 `[Character]` 数组。
- 使用滑动窗口 (Sliding Window) + 哈希表 (Dictionary) 的经典解法，时间复杂度 $O(n)$ 。

如需在 LeetCode Swift 环境中提交，只需保留 `Solution` 类即可。

矩阵

无

哈希表

1 两数之和

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        idx = {} # 创建一个空哈希表 (字典)
        for j, x in enumerate(nums): # x=nums[j]
            if target - x in idx: # 在左边找 nums[i], 满足 nums[i]+x=target
                return [idx[target - x], j] # 返回两个数的下标
            idx[x] = j # 保存 nums[j] 和 j
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/two-sum/solutions/2326193/dong-hua-cong-liang-shu-zhi-he-zhong-wo-0yvmj/>

来源：力扣 (LeetCode)

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设 `nums = [2,7,11,15]`, `target = 9`, 请将这段代码转换为swift版本，并给出完整调用代码。

当然可以！下面是将你提供的 Python 代码转换为 **Swift** 的版本，并附上完整的可运行调用示例。

✅ Swift 版本实现

```
class Solution {  
    func twoSum(_ nums: [Int], _ target: Int) -> [Int] {  
        var idx: [Int: Int] = [:] // 创建一个空哈希表 (字典), key 是数值, value 是下标  
        for (j, x) in nums.enumerated() { // x = nums[j]  
            let complement = target - x  
            if let i = idx[complement] { // 如果 complement 存在于哈希表中  
                return [i, j] // 返回两个下标  
            }  
            idx[x] = j // 将当前数值和下标存入哈希表  
        }  
        // 题目保证有解, 所以理论上不会执行到这里  
        return []  
    }  
}
```

✅ 完整调用示例（可直接在 Swift Playground 或 Xcode 中运行）

```
// 定义 Solution 类 (如上)  
class Solution {  
    func twoSum(_ nums: [Int], _ target: Int) -> [Int] {  
        var idx: [Int: Int] = [:]  
        for (j, x) in nums.enumerated() {  
            let complement = target - x  
            if let i = idx[complement] {  
                return [i, j]  
            }  
            idx[x] = j  
        }  
        return []  
    }  
}  
  
// 调用示例  
let solution = Solution()  
let nums = [2, 7, 11, 15]  
let target = 9  
let result = solution.twoSum(nums, target)  
print(result) // 输出: [0, 1]
```

🔍 说明

- Swift 中的字典 `Dictionary<Key, Value>` 对应 Python 的 `dict`。
- `nums.enumerated()` 返回一个 `(offset, element)` 的序列，等价于 Python 的 `enumerate(nums)`。
- 使用可选绑定 `if let i = idx[complement]` 来安全地检查键是否存在并获取值。
- 题目保证有唯一解，因此无需处理无解情况，但函数仍需返回 `[Int]`，故末尾加 `return []` 满足编译要求。

如有需要，也可以提供 **不使用类** 的函数式写法，或者适配 LeetCode 的提交格式（LeetCode 的 Swift 模板通常就是上述类形式）。

区间

无

栈

20 有效的括号

```
class Solution:
    def isValid(self, s: str) -> bool:
        if len(s) % 2: # s 长度必须是偶数
            return False
        mp = {'(': ')', '(': ')', '[': ']', '}'': '{'}
        st = []
        for c in s:
            if c not in mp: # c 是左括号
                st.append(c) # 入栈
            elif not st or st.pop() != mp[c]: # c 是右括号
                return False # 没有左括号，或者左括号类型不对
        return not st # 所有左括号必须匹配完毕
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/valid-parentheses/solutions/2809539/gua-hao-xiao-xiao-le-san-chong-li-yong-z-2xb3/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `s = "()"`，请将这段代码转换为swift版本，并给出完整调用代码。

下面是将你提供的 Python 代码转换为 **Swift** 的完整版本，包括 `Solution` 类和调用示例：

```
class Solution {
    func isValid(_ s: String) -> Bool {
        // 如果字符串长度是奇数，直接返回 false
        if s.count % 2 != 0 {
            return false
        }

        // 定义右括号到左括号的映射
        let mapping: [Character: Character] = [")": "(", "]"": "[", "}": "{"]

        var stack: [Character] = []

        for c in s {
            if !mapping.keys.contains(c) {
                // c 是左括号，入栈
                stack.append(c)
            } else {
                // c 是右括号
                if stack.isEmpty || stack.removeLast() != mapping[c] {
                    return false
                }
            }
        }

        // 最后栈必须为空才有效
        return stack.isEmpty
    }
}

// 调用示例
let solution = Solution()
let input = "()"
let result = solution.isValid(input)
print(result) // 输出: true
```

说明：

- Swift 中字符串遍历使用 `for c in s`，其中 `c` 是 `Character` 类型。
- 使用字典 `mapping` 来映射右括号到对应的左括号。
- 使用数组 `stack` 模拟栈操作：`append(_:)` 相当于 `push`，`removeLast()` 相当于 `pop`。
- 判断栈是否为空用 `isEmpty`。

这段代码在输入 `s = "()"` 时会输出 `true`，与原 Python 逻辑完全一致。

155 最小栈

```
class MinStack:
    def __init__(self):
        # 这里的 0 写成任意数都可以，反正用不到
        self.st = [(0, inf)] # 栈底哨兵

    def push(self, val: int) -> None:
        self.st.append((val, min(self.st[-1][1], val)))

    def pop(self) -> None:
        self.st.pop()

    def top(self) -> int:
        return self.st[-1][0]

    def getMin(self) -> int:
        return self.st[-1][1]
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/min-stack/solutions/2974438/ben-zhi-shi-wei-hu-qian-zhui-zui-xiao-zh-x0g8/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `["MinStack","push","push","push","getMin","pop","top","getMin"]` 和 `[[],[[-2],[0],[-3],[],[],[],[[[]]]]`，请将这段代码转换为 swift 版本，并给出完整调用代码。

下面是将你提供的 Python 版 **最小栈（MinStack）** 实现转换为 **Swift** 的完整代码，并附上针对输入：

```
["MinStack","push","push","push","getMin","pop","top","getMin"]
[[],[[-2],[0],[-3],[],[],[[[]]]]
```

的完整调用示例。

✅ Swift 版 MinStack 实现

```
import Foundation

class MinStack {
    private var stack: [(Int, Int)] // (value, currentMin)

    init() {
        // 使用哨兵，避免空栈判断；初始最小值设为 Int.max（相当于 Python 的 inf）
        stack = [(0, Int.max)]
    }

    func push(_ val: Int) {
        let currentMin = min(stack.last!.1, val)
        stack.append((val, currentMin))
    }

    func pop() {
        stack.removeLast()
    }

    func top() -> Int {
        return stack.last!.0
    }

    func getMin() -> Int {
        return stack.last!.1
    }
}
```

✅ 完整调用示例（模拟题目输入）

```
// 模拟输入操作
let ops = ["MinStack", "push", "push", "push", "getMin", "pop", "top", "getMin"]
let params: [[Int?]] = [[], [-2], [0], [-3], [], [], [], []]

var minStack: MinStack? = nil
var results: [Any?] = []

for i in 0..
```

🔍 说明

- `stack` 存储元组（当前值，当前最小值）。
- 初始哨兵 `(0, Int.max)` 确保第一次 `min(...)` 正确工作。
- 调用顺序严格按照 LeetCode 测试用例格式。
- 最终 `results` 中非 `nil` 的值就是 `getMin` 和 `top` 的返回结果。

✅ 运行上述代码后，控制台将输出：

```
[-3, 0, -2]
```

这与 LeetCode 示例一致：

- push -2 → 栈：[-2], min = -2
- push 0 → 栈：[-2, 0], min = -2
- push -3 → 栈：[-2, 0, -3], min = -3
- getMin → -3
- pop → 弹出 -3, 栈：[-2, 0]
- top → 0
- getMin → -2

如需进一步封装成 LeetCode 可提交格式，也可以提供。

链表

141 环形链表

```
class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        slow = fast = head # 乌龟和兔子同时从起点出发
        while fast and fast.next:
            slow = slow.next # 乌龟走一步
            fast = fast.next.next # 兔子走两步
            if fast is slow: # 兔子追上乌龟（套圈），说明有环
                return True
        return False # 访问到了链表末尾，无环
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/linked-list-cycle/solutions/1999269/mei-xiang-ming-bai-yi-ge-shi-pin-jiang-t-c4sw/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设 `head = [3,2,0,-4]`, `pos = 1`, 请将这段代码转换为swift版本, 并给出完整调用代码。

下面是将你提供的 Python 版本的 **环形链表检测算法（快慢指针法）** 转换为 **Swift** 的完整实现, 包括 `ListNode` 定义、`Solution` 类、以及一个完整的调用示例（模拟 `head = [3,2,0,-4]` 且 `pos = 1` 的有环链表）。

✅ Swift 完整代码

```
// Definition for singly-linked list.
public class ListNode {
    public var val: Int
    public var next: ListNode?
    public init(_ val: Int) {
        self.val = val
        self.next = nil
    }
}

class Solution {
    func hasCycle(_ head: ListNode?) -> Bool {
        var slow = head
        var fast = head

        while fast != nil && fast?.next != nil {
            slow = slow?.next
            fast = fast?.next?.next
            if slow === fast { // 注意：这里用 === 比较引用是否相同
                return true
            }
        }
        return false
    }
}

// 📌 构造测试用例：head = [3,2,0,-4], pos = 1 (即 -4 指向索引 1 的节点，值为 2)
let node3 = ListNode(3)
let node2 = ListNode(2)
let node0 = ListNode(0)
let nodeMinus4 = ListNode(-4)

node3.next = node2
node2.next = node0
node0.next = nodeMinus4
nodeMinus4.next = node2 // 形成环：-4 -> 2

// 调用
let solution = Solution()
let result = solution.hasCycle(node3)
print("Has cycle:", result) // 输出：Has cycle: true
```

🔍 说明:

- `===` 是 Swift 中用于比较两个对象是否是同一个实例（引用相等），对应 Python 中的 `is`。
- 链表构造部分手动创建了 4 个节点，并让最后一个节点（-4）指向索引 1 的节点（值为 2），从而形成环。
- 如果 `pos = -1`（无环），则 `nodeMinus4.next = nil` 即可。

如有需要，也可以封装一个通用的链表构造函数来根据数组和 `pos` 自动建环。是否需要我补充这个功能？

2 两数相加

```
class Solution:
    # l1 和 l2 为当前遍历的节点，carry 为进位
    def addTwoNumbers(self, l1: Optional[ListNode], l2: Optional[ListNode], carry=0)
-> Optional[ListNode]:
        if l1 is None and l2 is None and carry == 0: # 递归边界
            return None

        s = carry
        if l1:
            s += l1.val # 累加进位与节点值
            l1 = l1.next
        if l2:
            s += l2.val
            l2 = l2.next

        # s 除以 10 的余数为当前节点值，商为进位
        return ListNode(s % 10, self.addTwoNumbers(l1, l2, s // 10))
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/add-two-numbers/solutions/2327008/dong-hua-jian-ji-xie-fa-cong-di-gui-dao-oe0di/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设 `l1 = [2,4,3]`，`l2 = [5,6,4]`，请将这段代码转换为swift版本，并给出完整调用代码。

以下是将你提供的 Python 递归解法完整转换为 **Swift** 的版本，并附带针对 `l1 = [2,4,3]`、`l2 = [5,6,4]` 的完整可运行调用代码：

✅ 完整 Swift 代码（可在 Xcode Playground 中直接运行）：

```
// MARK: - 链表节点定义 (LeetCode 已内置, 此处为自包含)
public class ListNode {
    public var val: Int
    public var next: ListNode?
    public init() { self.val = 0; self.next = nil; }
    public init(_ val: Int) { self.val = val; self.next = nil; }
    public init(_ val: Int, _ next: ListNode?) { self.val = val; self.next = next; }
}

// MARK: - 解法类
class Solution {
    func addTwoNumbers(_ l1: ListNode?, _ l2: ListNode?, _ carry: Int = 0) ->
    ListNode? {
        // 递归终止条件: 两个链表都为空, 且无进位
        if l1 == nil && l2 == nil && carry == 0 {
            return nil
        }

        var sum = carry

        // 移动指针前先保存 next (避免在 if 中修改原变量)
        let nextL1 = l1?.next
        let nextL2 = l2?.next

        if let node = l1 { sum += node.val }
        if let node = l2 { sum += node.val }

        // 创建当前节点, 并递归构造后续节点
        return ListNode(sum % 10, addTwoNumbers(nextL1, nextL2, sum / 10))
    }
}

// MARK: - 辅助函数: 从数组构建链表
func buildList(from array: [Int]) -> ListNode? {
    guard !array.isEmpty else { return nil }
    let head = ListNode(array[0])
    var current = head
    for i in 1..
```

```
// MARK: - 辅助函数：将链表转为数组以便打印
func listToArray(_ head: ListNode?) -> [Int] {
    var result: [Int] = []
    var node = head
    while node != nil {
        result.append(node!.val)
        node = node!.next
    }
    return result
}

// MARK: - 主程序：测试用例
let l1Array = [2, 4, 3] // 表示数字 342（逆序）
let l2Array = [5, 6, 4] // 表示数字 465（逆序）

let l1 = buildList(from: l1Array)
let l2 = buildList(from: l2Array)

let solution = Solution()
let result = solution.addTwoNumbers(l1, l2)

print("l1 =", l1Array)
print("l2 =", l2Array)
print("Result =", listToArray(result)) // 应输出 [7, 0, 8]
```

执行结果：

```
l1 = [2, 4, 3]
l2 = [5, 6, 4]
Result = [7, 0, 8]
```

原理解释：

- 链表 [2,4,3] 表示 **342**（个位 → 百位）
 - 链表 [5,6,4] 表示 **465**
 - 相加：342 + 465 = **807**
 - 结果以逆序存储：**[7, 0, 8]**
-

💡 **提示：** 在 LeetCode 提交时，只需保留 `Solution` 类即可（平台已提供 `ListNode` 定义）。上述完整代码用于本地调试或 Playground 测试。

21 合并两个有序链表

```
class Solution:
    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
        cur = dummy = ListNode() # 用哨兵节点简化代码逻辑
        while list1 and list2:
            if list1.val < list2.val:
                cur.next = list1 # 把 list1 加到新链表中
                list1 = list1.next
            else: # 注：相等的情况加哪个节点都是可以的
                cur.next = list2 # 把 list2 加到新链表中
                list2 = list2.next
            cur = cur.next
        cur.next = list1 or list2 # 拼接剩余链表
        return dummy.next
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/merge-two-sorted-lists/solutions/2373691/liang-chong-fang-fa-die-dai-di-gui-pytho-wf75/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设 `list1 = [1,2,4]`，`list2 = [1,3,4]`，请将这段代码转换为swift版本，并给出完整调用代码。

下面是将你提供的 Python 代码转换为 **Swift** 的完整实现，包括 `ListNode` 的定义、`mergeTwoLists` 方法以及完整的调用示例。

✅ Swift 完整代码（含 ListNode 定义 + 调用）

```
// Definition for singly-linked list.
public class ListNode {
    public var val: Int
    public var next: ListNode?
    public init() { self.val = 0; self.next = nil; }
    public init(_ val: Int) { self.val = val; self.next = nil; }
    public init(_ val: Int, _ next: ListNode?) {
        self.val = val
        self.next = next
    }
}

class Solution {
    func mergeTwoLists(_ list1: ListNode?, _ list2: ListNode?) -> ListNode? {
        let dummy = ListNode(0) // 哨兵节点
        var cur = dummy
        var l1 = list1
        var l2 = list2

        while l1 != nil && l2 != nil {
            if l1!.val < l2!.val {
                cur.next = l1
                l1 = l1!.next
            } else {
                cur.next = l2
                l2 = l2!.next
            }
            cur = cur.next!
        }

        // 拼接剩余部分
        cur.next = l1 ?? l2

        return dummy.next
    }
}

// 📌 辅助函数：从数组构建链表
func buildList(from array: [Int]) -> ListNode? {
    guard !array.isEmpty else { return nil }
    let head = ListNode(array[0])
    var current = head
    for i in 1..
```

```

        current = current.next!
    }
    return head
}

// 📌 辅助函数：打印链表
func printList(_ head: ListNode?) {
    var node = head
    var values: [Int] = []
    while node != nil {
        values.append(node!.val)
        node = node!.next
    }
    print(values)
}

// 🖋️ 测试代码
let list1 = buildList(from: [1, 2, 4])
let list2 = buildList(from: [1, 3, 4])

let solution = Solution()
let merged = solution.mergeTwoLists(list1, list2)

print("合并后的链表:")
printList(merged) // 输出: [1, 1, 2, 3, 4, 4]

```

🔍 说明：

- Swift 中可选类型（Optional）使用 `?` 表示，所以需要小心处理 `nil`。
- 使用 `l1!` 是因为我们在 `while l1 != nil && l2 != nil` 条件下才进入循环，此时可以安全解包。
- `cur.next = l1 ?? l2` 利用了 Swift 的空合运算符，等价于 Python 中的 `list1 or list2`。
- 提供了辅助函数 `buildList` 和 `printList` 便于测试和验证。

运行上述代码后，输出为：

```

合并后的链表：
[1, 1, 2, 3, 4, 4]

```

这与 LeetCode 示例一致 ✅。

19 删除链表的倒数第 N 个结点

```
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) ->
Optional[ListNode]:
    # 由于可能会删除链表头部，用哨兵节点简化代码
    left = right = dummy = ListNode(next=head)
    for _ in range(n):
        right = right.next # 右指针先向右走 n 步
    while right.next:
        left = left.next
        right = right.next # 左右指针一起走
    left.next = left.next.next # 左指针的下一个节点就是倒数第 n 个节点
    return dummy.next
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/remove-nth-node-from-end-of-list/solutions/2004057/ru-he-shan-chu-jie-dian-liu-fen-zhong-ga-xpfs/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设 `head = [1,2,3,4,5]`，`n = 2`，请将这段代码转换为swift版本，并给出完整调用代码。

下面是将你提供的 Python 代码转换为 **Swift** 的完整实现，包括 `ListNode` 定义、`Solution` 类以及完整的调用示例。

✅ Swift 版本实现

```
// Definition for singly-linked list.
public class ListNode {
    public var val: Int
    public var next: ListNode?
    public init() { self.val = 0; self.next = nil; }
    public init(_ val: Int) { self.val = val; self.next = nil; }
    public init(_ val: Int, _ next: ListNode?) {
        self.val = val
        self.next = next
    }
}

class Solution {
    func removeNthFromEnd(_ head: ListNode?, _ n: Int) -> ListNode? {
        // 创建哨兵节点 (dummy node)
        let dummy = ListNode(0, head)
        var left: ListNode? = dummy
        var right: ListNode? = dummy

        // 右指针先走 n 步
        for _ in 0..
```

完整调用示例（构建链表 [1,2,3,4,5], n = 2）

```
// 构建链表 1 -> 2 -> 3 -> 4 -> 5
let node5 = ListNode(5)
let node4 = ListNode(4, node5)
let node3 = ListNode(3, node4)
let node2 = ListNode(2, node3)
let node1 = ListNode(1, node2)

let solution = Solution()
let newHead = solution.removeNthFromEnd(node1, 2)

// 打印结果：应输出 [1, 2, 3, 5]
func printList(_ head: ListNode?) {
    var current = head
    var values: [Int] = []
    while current != nil {
        values.append(current!.val)
        current = current?.next
    }
    print(values)
}

printList(newHead) // 输出: [1, 2, 3, 5]
```

说明

- 使用了 **双指针技巧**（快慢指针）：
 - `right` 先走 `n` 步；
 - 然后 `left` 和 `right` 同步前进，当 `right` 到达末尾时，`left` 恰好在倒数第 `n+1` 个点；
 - 此时删除 `left.next` 即可。
- 哨兵节点（dummy）避免了对头节点删除的特殊处理。

如有需要支持 LeetCode 提交格式，也可以去掉 `public` 关键字（LeetCode 环境中通常不需要）。

146 LRU 缓存

```
class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        # from collections import OrderedDict
        # OrderedDict = dict + 双向链表
        self.cache = OrderedDict() # key -> value

    def get(self, key: int) -> int:
        if key not in self.cache: # 没有这本书
            return -1
        # 有这本书, 把这本书抽出来, 放到最上面 (last=False 表示移到链表头)
        self.cache.move_to_end(key, last=False)
        return self.cache[key]

    def put(self, key: int, value: int) -> None:
        self.cache[key] = value # 添加 key value 或者更新 value
        # 把这本书抽出来, 放到最上面 (last=False 表示移到链表头)
        self.cache.move_to_end(key, last=False)
        if len(self.cache) > self.capacity: # 书太多了
            self.cache.popitem() # 去掉最后一本书
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/lru-cache/solutions/2456294/tu-jie-yi-zhang-tu-miao-dong-lrpythonja-czgt/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 ["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]、[[2], [1,1], [2,2], [1], [3,3], [2], [4,4], [1], [3], [4]], 请将这段代码转换为swift版本, 并给出完整调用代码。

下面是将你提供的 Python 版本的 LRU 缓存实现转换为 **Swift** 的完整代码, 并附上针对输入:

```
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1,1], [2,2], [1], [3,3], [2], [4,4], [1], [3], [4]]
```

的完整调用示例。

✅ Swift 实现（使用 `NSMutableDictionary` + 双向链表）

由于 Swift 标准库中没有类似 Python 的 `OrderedDict`，我们需要手动实现一个支持 **$O(1)$** 插入、删除、移动到头部 的结构。通常的做法是结合 哈希表 + 双向链表。

```
class LRUCache {

    private class Node {
        var key: Int
        var value: Int
        var prev: Node?
        var next: Node?

        init(key: Int, value: Int) {
            self.key = key
            self.value = value
        }
    }

    private let capacity: Int
    private var cache: [Int: Node] = [:]
    private let head: Node
    private let tail: Node

    init(_ capacity: Int) {
        self.capacity = capacity
        // 哨兵节点，简化边界处理
        head = Node(key: 0, value: 0)
        tail = Node(key: 0, value: 0)
        head.next = tail
        tail.prev = head
    }

    func get(_ key: Int) -> Int {
        guard let node = cache[key] else {
            return -1
        }
        // 移动到头部（最近使用）
        removeNode(node)
        addToHead(node)
        return node.value
    }

    func put(_ key: Int, _ value: Int) {
        if let existingNode = cache[key] {
            // 更新值
            existingNode.value = value
            removeNode(existingNode)
            addToHead(existingNode)
        } else {
            // 新增节点
        }
    }
}
```

```
        let newNode = Node(key: key, value: value)
        cache[key] = newNode
        addToHead(newNode)

        if cache.count > capacity {
            // 删除尾部（最久未使用）
            if let lastNode = tail.prev {
                removeNode(lastNode)
                cache[lastNode.key] = nil
            }
        }
    }
}

// 辅助方法：将节点添加到头部（head 之后）
private func addToHead(_ node: Node) {
    node.prev = head
    node.next = head.next
    head.next?.prev = node
    head.next = node
}

// 辅助方法：从链表中移除节点
private func removeNode(_ node: Node) {
    node.prev?.next = node.next
    node.next?.prev = node.prev
}
}
```

✅ 完整调用示例（模拟题目输入）

```
// 模拟输入
let ops = ["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
let params = [[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]

var lru: LRUCache? = nil
var results: [Any] = []

for i in 0..
```

✅ 预期输出

运行上述代码后，`results` 应为：

```
["null", "null", "null", 1, "null", -1, "null", -1, 3, 4]
```

解释：

- `get(1)` → 1 (存在)
 - `get(2)` → -1 (被 evict 了)
 - `get(1)` → -1 (又被 evict)
 - `get(3)` → 3
 - `get(4)` → 4
-

如有需要，也可以用 `NSMutableDictionary` 实现弱引用版本，但本题 key 是 `Int`，不需要考虑循环引用，直接用 `[Int: Node]` 即可。

如需进一步优化或封装，请告诉我！

二叉树

无

二叉树层次遍历

无

二叉搜索树

无

图

无

图的广度优先搜索

无

字典树

无

回溯

无

分治

无

Kadane 算法

无

二分查找

35 搜索插入位置

```
# 左闭右开区间写法
def lower_bound2(nums: List[int], target: int) -> int:
    left = 0
    right = len(nums) # 左闭右开区间 [left, right)
    while left < right: # 区间不为空
        # 循环不变量:
        # nums[left-1] < target
        # nums[right] >= target
        mid = (left + right) // 2
        if nums[mid] < target:
            left = mid + 1 # 范围缩小到 [mid+1, right)
        else:
            right = mid # 范围缩小到 [left, mid)
    return left # 或者 right
```

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        return lower_bound2(nums, target) # 选择其中一种写法即可
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/search-insert-position/solutions/2023391/er-fen-cha-zhao-zong-shi-xie-bu-dui-yi-g-nq23/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `nums = [1,3,5,6]`, `target = 5`, 请将这段代码转换为swift版本, 并给出完整调用代码。

以下是将你提供的 Python 二分查找代码（左闭右开区间写法）转换为 **Swift** 的完整实现，包括调用示例：

```

import Foundation

func lowerBound2(_ nums: [Int], _ target: Int) -> Int {
    var left = 0
    var right = nums.count // 左闭右开区间 [left, right)

    while left < right {
        // 循环不变量:
        // nums[left - 1] < target (如果 left > 0)
        // nums[right] >= target (如果 right < nums.count)
        let mid = (left + right) / 2
        if nums[mid] < target {
            left = mid + 1 // 范围缩小到 [mid+1, right)
        } else {
            right = mid // 范围缩小到 [left, mid)
        }
    }
    return left // 此时 left == right
}

class Solution {
    func searchInsert(_ nums: [Int], _ target: Int) -> Int {
        return lowerBound2(nums, target)
    }
}

// ===== 调用示例 =====
let nums = [1, 3, 5, 6]
let target = 5

let solution = Solution()
let result = solution.searchInsert(nums, target)

print("插入位置为: \(result)") // 输出: 插入位置为: 2

```

说明:

- Swift 中数组使用 `[Int]` 类型。
- 整数除法 `/` 在 Swift 中对整数自动向下取整（与 Python 的 `//` 行为一致）。
- 该实现严格遵循左闭右开区间 `[left, right)` 的二分查找逻辑。
- 对于输入 `nums = [1,3,5,6]`，`target = 5`，函数正确返回索引 `2`。

你可以将上述代码复制到 Xcode Playground 或 Swift 项目中直接运行。

34 在排序数组中查找元素的第一个和最后一个位置

```
class Solution:
    # lower_bound 返回最小的满足 nums[i] >= target 的下标 i
    # 如果数组为空, 或者所有数都 < target, 则返回 len(nums)
    # 要求 nums 是非递减的, 即 nums[i] <= nums[i + 1]
    def lower_bound(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) # 左闭右开区间 [left, right)
        while left < right: # 区间不为空
            # 循环不变量:
            # nums[left-1] < target
            # nums[right] >= target
            mid = (left + right) // 2
            if nums[mid] >= target:
                right = mid # 范围缩小到 [left, mid)
            else:
                left = mid + 1 # 范围缩小到 [mid+1, right)
        # 循环结束后 left = right
        # 此时 nums[left-1] < target 而 nums[left] = nums[right] >= target
        # 所以 left 就是第一个 >= target 的元素下标
        return left

    def searchRange(self, nums: List[int], target: int) -> List[int]:
        start = self.lower_bound(nums, target) # 选择其中一种写法即可
        if start == len(nums) or nums[start] != target:
            return [-1, -1] # nums 中没有 target
        # 如果 start 存在, 那么 end 必定存在
        end = self.lower_bound(nums, target + 1) - 1
        return [start, end]
```

作者: 灵茶山艾府

链接: <https://leetcode.cn/problems/find-first-and-last-position-of-element-in-sorted-array/solutions/1980196/er-fen-cha-zhao-zong-shi-xie-bu-dui-yi-g-t9l9/>

来源: 力扣 (LeetCode)

著作权归作者所有。商业转载请联系作者获得授权, 非商业转载请注明出处。

假设输入 `nums = [5,7,7,8,8,10]`, `target = 8`, 请将这段代码转换为swift版本, 并给出完整调用代码。

下面是将你提供的 Python 代码完整转换为 **Swift** 的版本, 并附上完整的调用示例。

✅ Swift 版本实现

```
import Foundation

class Solution {
    // lowerBound 返回最小的满足 nums[i] >= target 的下标 i
    // 如果数组为空, 或者所有数都 < target, 则返回 nums.count
    // 要求 nums 是非递减的
    func lowerBound(_ nums: [Int], _ target: Int) -> Int {
        var left = 0
        var right = nums.count // 左闭右开区间 [left, right)
        while left < right {
            let mid = (left + right) / 2
            if nums[mid] >= target {
                right = mid // 缩小到 [left, mid)
            } else {
                left = mid + 1 // 缩小到 [mid+1, right)
            }
        }
        return left
    }

    func searchRange(_ nums: [Int], _ target: Int) -> [Int] {
        let start = lowerBound(nums, target)
        // 检查是否真的存在 target
        if start == nums.count || nums[start] != target {
            return [-1, -1]
        }
        let end = lowerBound(nums, target + 1) - 1
        return [start, end]
    }
}
```

🖋️ 完整调用示例

```
let solution = Solution()
let nums = [5, 7, 7, 8, 8, 10]
let target = 8
let result = solution.searchRange(nums, target)
print(result) // 输出: [3, 4]
```

💡 补充说明

- Swift 中数组下标是 `Int` 类型，和 Python 一致。
- 二分查找使用的是 **左闭右开** 区间 `[left, right)`，这是非常常见且不容易出错的写法。
- `lowerBound(nums, target + 1)` 找的是第一个 **大于 target** 的位置，减 1 就是最后一个等于 `target` 的位置。

如有需要，也可以封装成扩展（Extension）或使用泛型，但针对 LeetCode 题目，上述写法已足够清晰高效。

堆

无

位运算

136 只出现一次的数字

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        return reduce(xor, nums)
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/single-number/solutions/2481594/li-yong-yi-huo-de-xing-zhi-fu-ti-dan-pyt-oizc/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `nums = [2,2,1]`，请将这段代码转换为 Swift 版本，并给出完整调用代码。

下面是将你提供的 Python 代码转换为 **Swift** 版本的实现，并附上完整的可运行调用代码。

✅ Swift 实现（使用异或 XOR）

```
import Foundation

class Solution {
    func singleNumber(_ nums: [Int]) -> Int {
        return nums.reduce(0, ^)
    }
}

// 示例调用
let nums = [2, 2, 1]
let solution = Solution()
let result = solution.singleNumber(nums)
print(result) // 输出: 1
```

🔍 说明：

- Swift 中的 `reduce` 方法与 Python 的 `functools.reduce` 类似。
- 异或运算符在 Swift 中是 `^`。
- 初始值设为 `0`，因为 $0 \wedge x == x$ ，符合异或的单位元性质。
- 对于输入 `[2, 2, 1]`：
 - $2 \wedge 2 = 0$
 - $0 \wedge 1 = 1$
 - 所以结果是 `1`，即只出现一次的数字。

这段代码可以直接复制到 Xcode Playground 或 Swift 项目中运行。

9 回文数

```
class Solution:
    def isPalindrome(self, x: int) -> bool:
        if x < 0 or x > 0 and x % 10 == 0:
            return False
        rev = 0
        while rev < x // 10:
            rev = rev * 10 + x % 10
            x //= 10
        return rev == x or rev == x // 10
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/palindrome-number/solutions/3682487/bi-guan-fang-ti-jie-shao-xun-huan-yi-ci-02nkc/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 $x = 121$ ，请将这段代码转换为swift版本，并给出完整调用代码。

下面是将你提供的 Python 代码（来自灵茶山艾府）转换为 **Swift** 版本的完整实现，并附带一个完整的调用示例。

✅ Swift 版本实现

```
class Solution {  
    func isPalindrome(_ x: Int) -> Bool {  
        // 负数不是回文，正数且末位为0（如10、100）也不是回文（除了0本身）  
        if x < 0 || (x > 0 && x % 10 == 0) {  
            return false  
        }  
  
        var rev = 0  
        var num = x  
  
        // 只反转一半数字  
        while rev < num / 10 {  
            rev = rev * 10 + num % 10  
            num /= 10  
        }  
  
        // 判断偶数位 (rev == num) 或奇数位 (rev == num / 10)  
        return rev == num || rev == num / 10  
    }  
}
```

🔪 完整调用示例（可直接在 Xcode Playground 中运行）

```
let solution = Solution()  
  
// 测试用例  
print(solution.isPalindrome(121)) // true  
print(solution.isPalindrome(-121)) // false  
print(solution.isPalindrome(10)) // false  
print(solution.isPalindrome(0)) // true  
print(solution.isPalindrome(1221)) // true  
print(solution.isPalindrome(12321)) // true
```

🔍 说明

- `x / 10` 在 Swift 中对整数是**向下取整除法**，和 Python 的 `//` 行为一致（只要 `x` 是正数）。
- 条件 `(x > 0 && x % 10 == 0)` 排除了像 `10`，`100`，`1230` 这类以 0 结尾但不是 0 的数，因为它们不可能是回文（首位不能为 0）。

- 算法只反转后半部分数字，避免了整数溢出风险，也满足“不转字符串”的进阶要求。

如有需要支持负数或其他边界情况，当前逻辑已涵盖。

66 加一

```
class Solution:
    def plusOne(self, digits: List[int]) -> List[int]:
        for i in range(len(digits) - 1, -1, -1):
            if digits[i] < 9:
                digits[i] += 1 # 进位
                return digits
            digits[i] = 0 # 进位数字的右边数字都变成 0

# digits 全是 9, 加一后变成 100...00
return [1] + digits
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/plus-one/solutions/3784551/jian-dan-ti-jian-dan-zuo-pythonjavaccgoj-6gb2/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `digits = [1,2,3]`，请将这段代码转换为swift版本，并给出完整调用代码。

下面是将你提供的 Python 代码转换为 **Swift** 的完整实现，并附上完整的调用示例。

✅ Swift 版本实现

```
import Foundation

class Solution {
    func plusOne(_ digits: [Int]) -> [Int] {
        var digits = digits // 创建可变副本

        // 从最后一位开始遍历
        for i in (0..
```

🔧 运行结果（针对输入 `[1,2,3]`）：

```
输入: [1, 2, 3]
输出: [1, 2, 4]
```

🔍 补充说明：

- Swift 中数组默认是值类型（`Array` 是结构体），所以传入函数后要赋值给一个 `var` 才能修改。
- 使用 `(0.. 来从后往前遍历索引，等价于 Python 的 range(len(digits)-1, -1, -1)。`

- 最后 `[1] + digits` 在 Swift 中也是合法的数组拼接语法。

如有其他测试用例或想封装成可运行的命令程序，也可以继续告诉我！

一维动态规划

70 爬楼梯

1. 启发思考：寻找子问题

```
# 会超时的递归代码
class Solution:
    def climbStairs(self, n: int) -> int:
        def dfs(i: int) -> int:
            if i <= 1: # 递归边界
                return 1
            return dfs(i - 1) + dfs(i - 2)
        return dfs(n)
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/climbing-stairs/solutions/2560716/jiao-ni-yi-bu-bu-si-kao-dong-tai-gui-hua-7zm1/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `n = 2`，请将这段代码转换为swift版本，并给出完整调用代码。

以下是将你提供的 Python 递归代码（会超时版本）转换为 **Swift** 的完整实现，包括调用示例：

```

class Solution {
    func climbStairs(_ n: Int) -> Int {
        func dfs(_ i: Int) -> Int {
            if i <= 1 {
                return 1
            }
            return dfs(i - 1) + dfs(i - 2)
        }
        return dfs(n)
    }
}

// 调用示例
let solution = Solution()
let result = solution.climbStairs(2)
print(result) // 输出: 2

```

说明:

- Swift 中函数使用 `func` 关键字定义。
- 嵌套函数（如 `dfs`）在 Swift 中是完全支持的。
- 条件判断和递归逻辑与 Python 版本一一对应。
- 对于 `n = 2`，输出结果为 `2`，符合题目示例。

⚠ 注意：这段代码虽然逻辑正确，但由于是朴素递归，**时间复杂度为 $O(2^n)$** ，在 `n` 较大时（如 `n > 40`）会非常慢甚至超时。实际应用中应使用记忆化搜索或动态规划优化。但根据你的要求，这里保留了原始的“会超时”递归结构。

2. 递归 + 记录返回值 = 记忆化搜索

```

class Solution:
    def climbStairs(self, n: int) -> int:
        @cache # 缓存装饰器，避免重复计算 dfs 的结果
        def dfs(i: int) -> int:
            if i <= 1: # 递归边界
                return 1
            return dfs(i - 1) + dfs(i - 2)
        return dfs(n)

```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/climbing-stairs/solutions/2560716/jiao-ni-yi-bu-bu-si-kao-dong-tai-gui-hua-7zm1/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 $n = 2$ ，请将这段代码转换为 Swift 版本，并给出完整调用代码。

在 Swift 中，标准库 **没有内置的 @cache 装饰器**（像 Python 的 `functools.cache` 那样），但我们可以通过手动使用字典（`Dictionary`）来实现**记忆化递归（Memoization）**，从而避免重复计算。

下面是将你提供的带缓存的 Python 代码转换为 **Swift 版本** 的完整实现，并包含调用示例：

```
class Solution {
    func climbStairs(_ n: Int) -> Int {
        var memo: [Int: Int] = [:] // 用于缓存已计算的结果

        func dfs(_ i: Int) -> Int {
            if i <= 1 {
                return 1
            }
            if let cached = memo[i] {
                return cached
            }
            let result = dfs(i - 1) + dfs(i - 2)
            memo[i] = result
            return result
        }

        return dfs(n)
    }
}

// 调用示例
let solution = Solution()
let result = solution.climbStairs(2)
print(result) // 输出: 2
```

说明：

- 使用 `memo` 字典存储已经计算过的 `dfs(i)` 结果。
- 每次进入 `dfs` 时先检查是否已缓存，若有则直接返回，避免重复递归。
- 时间复杂度优化为 **$O(n)$** ，空间复杂度为 **$O(n)$** （递归栈 + 缓存）。
- 对于输入 $n = 2$ ，输出为 `2`，与题目一致。

✅ 这是 Swift 中实现记忆化搜索的标准方式。虽然不如 Python 的 `@cache` 简洁，但功能完全等价。

多维动态规划

5 最长回文子串

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        n = len(s)
        ans_left = ans_right = 0

        # 奇回文串
        for i in range(n):
            l = r = i
            while l >= 0 and r < n and s[l] == s[r]:
                l -= 1
                r += 1
            # 循环结束后, s[l+1] 到 s[r-1] 是回文串
            if r - l - 1 > ans_right - ans_left:
                ans_left, ans_right = l + 1, r # 左闭右开区间

        # 偶回文串
        for i in range(n - 1):
            l, r = i, i + 1
            while l >= 0 and r < n and s[l] == s[r]:
                l -= 1
                r += 1
            if r - l - 1 > ans_right - ans_left:
                ans_left, ans_right = l + 1, r # 左闭右开区间

        return s[ans_left: ans_right]
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/longest-palindromic-substring/solutions/2958179/mo-ban-on-manacher-suan-fa-pythonjavacgo-t6cx/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `s = "babad"`，请将这段代码转换为swift版本，并给出完整调用代码。

以下是将你提供的 Python 代码转换为 **Swift** 的完整版本，并附带调用示例：

✅ Swift 版本实现

```
class Solution {  
    func longestPalindrome(_ s: String) -> String {  
        let chars = Array(s)  
        let n = chars.count  
        var ansLeft = 0  
        var ansRight = 0  
  
        // 奇数长度回文 (以 i 为中心)  
        for i in 0..  
n {  
            var l = i  
            var r = i  
            while l >= 0 && r < n && chars[l] == chars[r] {  
                l -= 1  
                r += 1  
            }  
            // 当前回文长度为 (r - l - 1)  
            if r - l - 1 > ansRight - ansLeft {  
                ansLeft = l + 1  
                ansRight = r  
            }  
        }  
  
        // 偶数长度回文 (以 i 和 i+1 为中心)  
        for i in 0..  
(n - 1) {  
            var l = i  
            var r = i + 1  
            while l >= 0 && r < n && chars[l] == chars[r] {  
                l -= 1  
                r += 1  
            }  
            if r - l - 1 > ansRight - ansLeft {  
                ansLeft = l + 1  
                ansRight = r  
            }  
        }  
  
        return String(chars[ansLeft..  
ansRight])  
    }  
}
```

完整调用示例

```
let solution = Solution()
let input = "babad"
let result = solution.longestPalindrome(input)
print("最长回文子串是: \(result)") // 输出可能是 "bab" 或 "aba"
```

💡 注意：对于 "babad"，存在多个合法答案（如 "bab" 或 "aba"），只要返回其中之一即为正确。

补充说明

- Swift 中 `String` 不支持直接下标访问字符（不像 Python），所以先转成 `[Character]` 数组。
- 使用左闭右开区间 `ansLeft..ansRight` 来切片字符串，符合 Swift 的习惯。
- 时间复杂度： $O(n^2)$ ，空间复杂度： $O(n)$ （因为转成了数组）。

如有需要，也可以进一步优化为 Manacher 算法（线性时间），但此解法清晰易懂，适合大多数场景。

Other

83 删除排序链表中的重复元素

```
class Solution:
    def deleteDuplicates(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None:
            return None
        cur = head
        while cur.next: # 看看下个节点.....
            if cur.next.val == cur.val: # 和我一样，删！
                cur.next = cur.next.next
            else: # 和我不一样，移动到下个节点
                cur = cur.next
        return head
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/remove-duplicates-from-sorted-list/solutions/2004062/ru-he-qu-zhong-yi-ge-shi-pin-jiang-tou-p-98g7/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设输入 `head = [1,1,2]`，请将这段代码转换为swift版本，并给出完整调用代码。

下面是将你提供的 Python 代码转换为 **Swift** 版本的实现，并附上完整的调用代码（包括 `ListNode` 定义、测试用例和运行示例）。

✅ Swift 实现（LeetCode 风格）

```
// Definition for singly-linked list.
public class ListNode {
    public var val: Int
    public var next: ListNode?
    public init() { self.val = 0; self.next = nil; }
    public init(_ val: Int) { self.val = val; self.next = nil; }
    public init(_ val: Int, _ next: ListNode?) {
        self.val = val
        self.next = next
    }
}

class Solution {
    func deleteDuplicates(_ head: ListNode?) -> ListNode? {
        guard let head = head else { return nil }
        var cur: ListNode? = head
        while cur?.next != nil {
            if cur!.next!.val == cur!.val {
                // 跳过重复节点
                cur!.next = cur!.next!.next
            } else {
                // 移动到下一个不同值的节点
                cur = cur!.next
            }
        }
        return head
    }
}
```

✅ 完整调用示例（可直接在 Playground 或 LeetCode 测试）

```
// 构建链表 [1, 1, 2]
let node3 = ListNode(2)
let node2 = ListNode(1, node3)
let node1 = ListNode(1, node2)

let solution = Solution()
let result = solution.deleteDuplicates(node1)

// 打印结果链表
func printList(_ head: ListNode?) {
    var current = head
    var values: [Int] = []
    while current != nil {
        values.append(current!.val)
        current = current!.next
    }
    print(values) // 应输出 [1, 2]
}

printList(result)
```

🔍 说明

- 输入 `[1,1,2]` 经过去重后应为 `[1,2]`。
- 该算法 **原地修改链表**，时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。
- Swift 中使用了可选绑定和强制解包（因题目保证结构有效），在实际工程中可进一步增加安全检查。

如需支持更多测试用例（如 `[1,1,2,3,3]` → `[1,2,3]`），只需调整构建链表部分即可。需要我帮你写一个通用的数组转链表函数吗？

24 两两交换链表中的节点

```
class Solution:
    def swapPairs(self, head: Optional[ListNode]) -> Optional[ListNode]:
        node0 = dummy = ListNode(next=head) # 用哨兵节点简化代码逻辑
        node1 = head
        while node1 and node1.next: # 至少有两个节点
            node2 = node1.next
            node3 = node2.next

            node0.next = node2 # 0 -> 2
            node2.next = node1 # 2 -> 1
            node1.next = node3 # 1 -> 3

            node0 = node1 # 下一轮交换, 0 是 1
            node1 = node3 # 下一轮交换, 1 是 3
        return dummy.next # 返回新链表的头节点
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/swap-nodes-in-pairs/solutions/2374872/tu-jie-die-dai-di-gui-yi-zhang-tu-miao-d-51ap/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设 head = [1,2,3,4]，请将这段代码转换为swift版本，并给出完整调用代码。

下面是将你提供的 Python 代码转换为 **Swift** 的版本，并附上完整的调用示例（包括 `ListNode` 定义和测试用例）。

✅ Swift 版本实现

```
// Definition for singly-linked list.
public class ListNode {
    public var val: Int
    public var next: ListNode?
    public init() { self.val = 0; self.next = nil; }
    public init(_ val: Int) { self.val = val; self.next = nil; }
    public init(_ val: Int, _ next: ListNode?) {
        self.val = val
        self.next = next
    }
}

class Solution {
    func swapPairs(_ head: ListNode?) -> ListNode? {
        let dummy = ListNode(0, head) // 哨兵节点
        var node0: ListNode? = dummy
        var node1 = head

        while node1 != nil && node1?.next != nil {
            let node2 = node1?.next
            let node3 = node2?.next

            // 交换 node1 和 node2
            node0?.next = node2           // 0 -> 2
            node2?.next = node1           // 2 -> 1
            node1?.next = node3           // 1 -> 3

            // 移动指针到下一轮
            node0 = node1                 // 下一轮的前驱是当前的 node1
            node1 = node3                 // 下一轮从 node3 开始
        }

        return dummy.next
    }
}
```

✅ 完整调用示例（用于测试）

```
// 构建链表 [1,2,3,4]
let node4 = ListNode(4)
let node3 = ListNode(3, node4)
let node2 = ListNode(2, node3)
let node1 = ListNode(1, node2)

let solution = Solution()
let newHead = solution.swapPairs(node1)

// 打印结果
func printList(_ head: ListNode?) {
    var current = head
    var values: [Int] = []
    while current != nil {
        values.append(current!.val)
        current = current?.next
    }
    print(values) // 应输出 [2, 1, 4, 3]
}

printList(newHead)
```



输出结果

运行上述代码后，控制台将输出：

```
[2, 1, 4, 3]
```

这表示成功将原链表 `[1 → 2 → 3 → 4]` 中的节点两两交换，得到 `[2 → 1 → 4 → 3]`。

如有需要支持 LeetCode 提交格式，也可以只保留 `Solution` 类和 `ListNode` 定义（LeetCode 已内置 `ListNode`，实际提交时可省略）。

206 反转链表

```
class Solution:
    # 首先「递」到链表末尾，把末尾节点作为新链表的头节点 rev_head
    # 然后在「归」的过程中，把经过的节点依次插在新链表的末尾（尾插法）
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        # 判断 head is None 是为了兼容一开始链表就是空的情况
        if head is None or head.next is None:
            return head # 链表末尾，即下面的 rev_head
        rev_head = self.reverseList(head.next) # 「递」到链表末尾，拿到新链表的头节点
        tail = head.next # 在「归」的过程中，head.next 就是新链表的末尾
        tail.next = head # 把 head 插在新链表的末尾
        head.next = None # 如果不写这行，新链表的末尾两个节点成环，这俩节点互相指向对方
        return rev_head
```

作者：灵茶山艾府

链接：<https://leetcode.cn/problems/reverse-linked-list/solutions/1992225/you-xie-cuo-liao-yi-ge-shi-pin-jiang-tou-o5zy/>

来源：力扣（LeetCode）

著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

假设 `head = [1,2,3,4,5]`，请将这段代码转换为swift版本，并给出完整调用代码。

以下是将你提供的 **Python 递归反转链表代码** 完整转换为 **Swift 版本** 的实现，并附带完整的调用示例（包括链表构建、反转和打印），适用于 `head = [1,2,3,4,5]`。

✅ 完整 Swift 代码（可直接在 Swift Playground 中运行）：

```
import Foundation

// Definition for singly-linked list.
public class ListNode {
    public var val: Int
    public var next: ListNode?

    public init() { self.val = 0; self.next = nil; }
    public init(_ val: Int) { self.val = val; self.next = nil; }
    public init(_ val: Int, _ next: ListNode?) {
        self.val = val
        self.next = next
    }
}

class Solution {
    func reverseList(_ head: ListNode?) -> ListNode? {
        // 基线条件：空链表或只有一个节点，直接返回
        if head == nil || head?.next == nil {
            return head
        }

        // 递归：先「递」到末尾，获取反转后的新头节点
        let revHead = reverseList(head?.next)

        // 「归」的过程中：
        let tail = head?.next // 此时 tail 是已反转部分的最后一个节点
        tail?.next = head     // 把当前 head 接到 tail 后面（尾插）
        head?.next = nil      // 断开原连接，防止成环

        return revHead        // 返回新的头节点（始终是原链表的最后一个节点）
    }
}

// Helper: 将数组转为链表
func buildLinkedList(from array: [Int]) -> ListNode? {
    guard !array.isEmpty else { return nil }
    let head = ListNode(array[0])
    var current = head
    for i in 1..
```

```

}

// Helper: 将链表转为数组 (用于打印)
func linkedListToArray(_ head: ListNode?) -> [Int] {
    var result: [Int] = []
    var node = head
    while node != nil {
        result.append(node!.val)
        node = node!.next
    }
    return result
}

// -----
// 主程序: 测试 head = [1,2,3,4,5]
// -----

let inputArray = [1, 2, 3, 4, 5]
let head = buildLinkedList(from: inputArray)

print("Original list: \(linkedListToArray(head))")

let solution = Solution()
let reversedHead = solution.reverseList(head)

print("Reversed list: \(linkedListToArray(reversedHead))")

```

运行输出:

```

Original list: [1, 2, 3, 4, 5]
Reversed list: [5, 4, 3, 2, 1]

```

逻辑说明 (与原 Python 一致):

- 递: 一直递归到 5 (此时 `head.next == nil`), 返回 5 作为 `revHead`。
- 归:
 - 在 4 层: `tail = 5` → `5.next = 4`, `4.next = nil` → 链表变为 `5 → 4`
 - 在 3 层: `tail = 4` → `4.next = 3`, `3.next = nil` → 链表变为 `5 → 4 → 3`
 -直到回到最外层, 最终得到 `5 → 4 → 3 → 2 → 1`

⚠️ 关键点: `head?.next = nil` 必须设置, 否则例如 `4.next` 原本指向 5, 而 `5.next` 又被设为 4, 就会形成环。

这段代码完全忠实于原作者的递归思路, 并适配了 Swift 的可选类型语法。可直接用于 LeetCode Swift 环境或本地 Playground 测试。